

§ 5.2 编码树

本章将以通讯编码算法的实现作为二叉树的应用实例。通讯理论中的一个基本问题是，如何在尽可能低的成本下，以尽可能高的速度，尽可能忠实地实现信息在空间和时间上的复制与转移。在现代通讯技术中，无论采用电、磁、光或其它任何形式，在信道上传递的信息大多以二进制比特的形式表示和存在，而每一个具体的编码方案都对应于一棵二叉编码树。

5.2.1 二进制编码

在加载到信道上之前，信息被转换为二进制形式的过程称作编码（encoding）；反之，经信道抵达目标后再由二进制编码恢复原始信息的过程称作解码（decoding）。

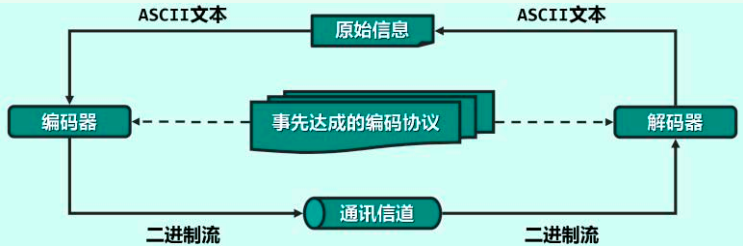


图5.7 完整的通讯过程由预处理、编码和解码阶段组成

如图5.7所示，编码和解码的任务分别由发送方和接收方分别独立完成，故在开始通讯之前，双方应已经以某种形式，就编码规则达成过共同的约定或协议。

■ 生成编码表

原始信息的基本组成单位称作字符，它们都来自于某一特定的有限集合 Σ ，也称作字符集（alphabet）。而以二进制形式承载的信息，都可表示为来自编码表 $\Gamma = \{ 0, 1 \}^*$ 的某一特定二进制串。从这个角度理解，每一编码表都是从字符集 Σ 到编码表 Γ 的一个单射，编码就是对信息文本中各字符逐个实施这一映射的过程，而解码则是逆向映射的过程。

表5.1 $\Sigma = \{ 'A', 'E', 'G', 'M', 'S' \}$ 的一份二进制编码表

字符	A	E	G	M	S
编码	00	01	10	110	111

表5.1即为二进制编码表的实例。编码表一旦制定，信息的发送方与接收方之间也就建立起了一个约定与默契，从而使得独立的编码与解码成为可能。

■ 二进制编码

现在，所谓编码就是对于任意给定的文本，通过查阅编码表逐一将其中的字符转译为二进制编码，这些编码依次串接起来即得到了全文的编码。比如若待编码文本为"MESSAGE"，则根据由表5.1确定的编码方案，对应的二进制编码串应为"110⁰¹111¹¹¹00¹⁰01"^①。

表5.2 二进制解码过程

二进制编码	当前匹配字符	解出原文
11001111111001001	M	M
011111111001001	E	ME
111111001001	S	MES
111001001	S	MESS
001001	A	MESSA
1001	G	MESSAG
01	E	MESSAGE

① 这里对各比特位做了适当的上下移位，以便读者区分各字符编码串的范围；在实际编码中，它们并无“高度”的区别

■ 二进制解码

由编码器生成的二进制流经信道送达之后，接收方可以按照事先约定的编码表（表5.1），依次扫描各比特位，并经匹配逐一转译出各字符，从而最终恢复出原始的文本。

仍以二进制编码串"110⁰¹111¹¹¹00¹⁰01"为例，其解码过程如表5.2所示。

■ 解码歧义

请注意，编码方案确定之后，尽管编码结果必然确定，但解码过程和结果却不见得唯一。

表5.3 $\Sigma = \{ 'A', 'E', 'G', 'M', 'S' \}$ 的另一份编码表

字 符	A	E	G	M	S
编码	00	01	10	11	111

比如，上述字符集 Σ 的另一编码方案如表5.3所示，与表5.1的差异在于，字符'M'的编码由"110"改为"11"。此时，原始文本"MESSAGE"经编码得到二进制编码串"11⁰¹111¹¹¹00¹⁰01"，但如表5.4左侧和右侧所示，解码方法却至少有两种。

表5.4 按照表5.3“确定”的编码协议，可能有多种解码结果

二进制编码	当前匹配字符	解出原文	二进制编码	当前匹配字符	解出原文
1101111111001001	M	M	1101111111001001	M	M
011111111001001	E	ME	011111111001001	E	ME
111111001001	S	MES	111111001001	M	MEM
111001001	S	MESS	1111001001	M	MEMM
001001	A	MESSA	11001001	M	MEMMM
1001	G	MESSAG	001001	A	MEMMMA
01	E	MESSAGE	1001	G	MEMMMAG
			01	E	MEMMMAGE

进一步推敲之后不难发现，按照这份编码表，有时甚至还会出现无法完成解码的情况。

■ 前缀无歧义编码

解码过程之所以会出现上述歧义甚至错误，根源在于编码表制订不当。这里的解码算法采用的是，按顺序对信息比特流做子串匹配的策略，因此为消除匹配的歧义性，任何两个原始字符所对应的二进制编码串，相互都不得是前缀。比如在表5.3中，字符'M'的编码（"11"）即为字符'S'的编码（"111"）的前缀，于是编码串"111111"既可以解释为：

"SS" = "111¹¹¹"

也可以解释为

"MMM" = "11¹¹11"

反过来，只要各字符的编码串互不为前缀，则即便出现无法解码的错误，也绝对不致歧义。这类编码方案即所谓的“前缀无歧义编码”（prefix-free code），简称PFC编码。此类编码算法，可以明确地将二进制编码串，分割为一系列与各原始字符对应的片段，从而实现无歧义的解码。得益于这一特点，此类算法在整个解码过程中，对信息比特流的扫描不必回溯。

那么，PFC编码的以上特点，可否直观解释？从算法角度看，PFC编码与解码过程，又该如何准确描述？从数据结构角度看，这些过程的实现，需要借助哪些功能接口？支持这些接口的数据结构，又该如何高效率地实现？以下以二叉树结构为模型，逐步解答这些疑问。

5.2.2 二叉编码树

■ 根通路与节点编码

任一编码方案都可描述为一棵二叉树：从根节点出发，每次向左（右）都对应于一个0（1）比特位。于是如图5.8所示，由从根节点到每个节点的唯一通路，可以为各节点 v 赋予一个互异的二进制串，称作根通路串（root path string），记作 $\text{rps}(v)$ 。当然， $|\text{rps}(v)| = \text{depth}(v)$ 就是 v 的深度。

若将 Σ 中的字符分别映射至二叉树的节点，则字符 x 的二进制编码串即可取作 $\text{rps}(v(x))$ 。以下，在不致引起混淆的前提下，不再区分字符 x 和与之对应的节点 $v(x)$ 。于是， $\text{rps}(v(x))$ 可简记作 $\text{rps}(x)$ ； $\text{depth}(v(x))$ 可简记作 $\text{depth}(x)$ 。

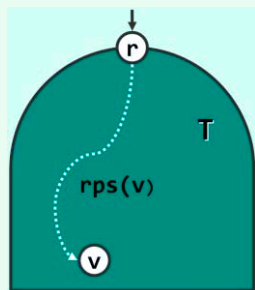


图5.8 二叉树中每个节点都由根通路串唯一确定

■ PFC编码树

仍以字符集 $\Sigma = \{ 'A', 'E', 'G', 'M', 'S' \}$ 为例，表5.1、表5.3所定义的编码方案分别对应于如图5.9左、右所示的二叉编码树。

易见， $\text{rps}(u)$ 是 $\text{rps}(v)$ 的前缀，当且仅当节点 u 是 v 的祖先，故表5.3中编码方案导致解码歧义的根本在于，在其编码树（图5.9(b)）中字符'M'是'S'的父亲。

反之，只要所有字符都对应于叶节点，歧义现象即自然消除——这也是实现PFC编码的简明策略。比如，图5.9(a)即为一种可行的PFC编码方案。

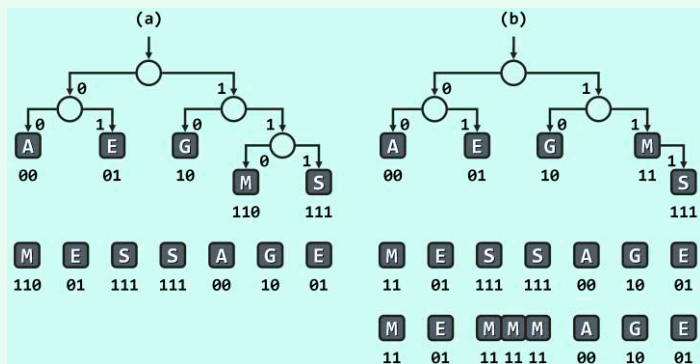


图5.9 $\Sigma = \{ 'A', 'E', 'G', 'M', 'S' \}$ 两种编码方案对应的二叉编码树

■ 基于PFC编码树的解码

反过来，依据PFC编码树可便捷地完成编码串的解码。依然以图5.9(a)中编码树为例，设对编码串“110⁰¹111¹¹¹00¹⁰01”解码。从前向后扫描该串，同时在树中相应移动。起始时从树根出发，视各比特位的取值相应地向左或右深入下一层，直到抵达叶节点。比如，在扫描过前三位“110”后将抵达叶节点'M'。此时，可以输出其对应的字符'M'，然后重新回到树根，并继续扫描编码串的剩余部分。比如，再经过接下来的两位“01”后将抵达叶节点'E'，同样地输出字符'E'并回到树根。如此迭代，即可无歧义地解析出原文中的所有字符（习题[5-6]）。

实际上，这一解码过程甚至可以在二进制编码串的接收过程中实时进行，而不必等到所有比特位都到达之后才开始，因此这类算法属于在线算法。

■ PFC编码树的构造

由上可见，PFC编码方案可由PFC编码树来描述，由编码树不仅可以快速生成编码表，而且直接支持高效的解码。那么，任意给定一个字符集 Σ ，如何构造出PFC编码方案呢？

为此，需要首先解决二叉树本身作为数据结构的描述和实现问题。

§ 5.3 二叉树的实现

作为图的特殊形式，二叉树的基本组成单元是节点与边；作为数据结构，其基本的组成实体是二叉树节点（binary tree node），而边则对应于节点之间的相互引用。

5.3.1 二叉树节点

■ BinNode模板类

以二叉树节点BinNode模板类，可定义如代码5.1所示。

```

1 #define BinNodePosi(T) BinNode<T>* //节点位置
2 #define stature(p) ((p) ? (p)->height : -1) //节点高度（与“空树高度为-1”的约定相统一）
3 typedef enum { RB_RED, RB_BLACK } RBColor; //节点颜色
4
5 template <typename T> struct BinNode { //二叉树节点模板类
6 // 成员（为简化描述起见统一开放，读者可根据需要进一步封装）
7     T data; //数值
8     BinNodePosi(T) parent; BinNodePosi(T) lChild; BinNodePosi(T) rChild; //父节点及左、右孩子
9     int height; //高度（通用）
10    int npl; //Null Path Length（左式堆，也可直接用height代替）
11    RBColor color; //颜色（红黑树）
12 // 构造函数
13    BinNode() : parent(NULL), lChild(NULL), rChild(NULL), height(0), npl(1), color(RB_RED) { }
14    BinNode(T e, BinNodePosi(T) p = NULL, BinNodePosi(T) lc = NULL, BinNodePosi(T) rc = NULL,
15            int h = 0, int l = 1, RBColor c = RB_RED)
16        : data(e), parent(p), lChild(lc), rChild(rc), height(h), npl(l), color(c) { }
17 // 操作接口
18    int size(); //统计当前节点后代总数，亦即以其为根的子树的规模
19    BinNodePosi(T) insertAsLC(T const&); //作为当前节点的左孩子插入新节点
20    BinNodePosi(T) insertAsRC(T const&); //作为当前节点的右孩子插入新节点
21    BinNodePosi(T) succ(); //取当前节点的直接后继
22    template <typename VST> void travLevel(VST&); //子树层次遍历
23    template <typename VST> void travPre(VST&); //子树先序遍历
24    template <typename VST> void travIn(VST&); //子树中序遍历
25    template <typename VST> void travPost(VST&); //子树后序遍历
26 // 比较器、判等器（各列其一，其余自行补充）
27    bool operator<(BinNode const& bn) { return data < bn.data; } //小于
28    bool operator==(BinNode const& bn) { return data == bn.data; } //等于
29 };

```

代码5.1 二叉树节点模板类BinNode

为简化起见，这里也未做严格的封装。通过宏BinNodePosi来指代节点位置，可以简化后续代码描述；通过定义宏stature，则可以保证从节点返回的高度值，能够与“空树高度为-1”的约定相统一。